EventSphere - Event Management and Ticket Booking System

Member 1: Jalal Haider - i228755 **Member 2: Umer Abdullah - i221349**

APRIL 10, 2026

EventSphere

1. Project Information

Project Title: EventSphere - Event Management and Ticket Booking System

2. Group Details (Strictly 2 Members)

Sr. No	Name	Roll Number	Email
1	Jalal Haider	i228755	To be added
2	Umer Abdullah	i221349	To be added

3. Project Overview

Problem Statement

Event organizers often rely on scattered tools for event posting, attendee registration, ticket sales, payment verification, venue communication, and attendee updates. This creates poor coordination, delayed confirmations, manual record keeping, and weak visibility into bookings, revenue, and attendee engagement.

Proposed Solution

EventSphere will be a MERN-based web application with two major panels: an Admin/Organizer Panel and a User/Attendee Panel. The system will allow administrators to create and manage events, categories, venues, tickets, bookings, and announcements, while users can browse events, search and filter listings, reserve tickets, make secure online payments, and track their booking history. The system will integrate Stripe Checkout for online payments, Google Maps Places API for venue search/location assistance, and SendGrid for transactional emails such as booking confirmations and reminders.

Target Users

- Event organizers / administrators
- General users / attendees
- University societies, clubs, or small event businesses
- Support or venue coordination staff (optional future role)

4. Technology Stack

Frontend: React.js, React Router, Redux Toolkit, Axios, Tailwind CSS, React Hook Form, Chart.js / Recharts

Backend: Node.js, Express.js

Database: MongoDB with Mongoose

Other Tools: JWT, bcrypt, Multer / Cloudinary (optional for image uploads), Socket.io (optional real-time updates), Postman, GitHub

External APIs (Mandatory): Stripe Checkout API for online payments, Google Maps Places API for venue search/autocomplete, SendGrid Email API for booking confirmations and reminders

5. Figma Design Prototype (MANDATORY)

The Figma prototype will cover both Admin and User panels. The layout will follow a responsive dashboard pattern with a top navigation bar, left sidebar for admin modules, and card-based event browsing for users. The UX will prioritize simple navigation, clear CTAs, readable forms, and mobile-friendly booking flow.

5.1 User Panel Screens

1. Landing Page / Home - hero section, featured events, category shortcuts, search bar
2. Login / Signup - clean authentication forms with validation and password visibility toggle
3. Browse Events - event cards, search input, date/category/price/location filters
4. Event Details - event description, organizer info, venue map snippet, ticket options, available seats
5. Checkout / Payment - attendee details, order summary, Stripe payment button / checkout redirect
6. Booking Success / Failure Screen - payment result, booking reference, download ticket CTA
7. My Bookings - list of purchased tickets with status, date, payment status, ticket download button
8. Profile Settings - user data update, password change, saved preferences
9. Notifications / Announcements - booking reminders and organizer notices
10. Contact / Help Screen - FAQs and support request form

5.2 Admin Panel Screens

1. Admin Login
2. Dashboard / Home - KPIs such as total events, total bookings, total users, revenue summary
3. Manage Events (CRUD) - create, read, update, delete events
4. Manage Categories (CRUD)
5. Manage Venues (CRUD) - including Google Maps assisted location entry
6. Manage Tickets / Pricing - ticket types, seat counts, price rules
7. Manage Users - view attendees, role handling, status control
8. Manage Bookings - booking verification, refund status, attendance status
9. Payments / Revenue Screen - Stripe payment records, revenue stats, refund monitoring
10. Search & Filter UI - across events, bookings, users, payment status, categories
11. Announcement / Email Broadcast Screen - send updates using SendGrid
12. Reports / Analytics Screen - charts for bookings, event popularity, and revenue
13. Optional Real-Time Monitor - live ticket sales and recent bookings using Socket.io

5.3 UX Decisions

1. User panel will use card-based browsing with clear filters and simple ticket purchase flow.
2. Admin panel will use a sidebar layout with data tables, modals, and reusable CRUD forms.
3. Forms will include inline validation to reduce errors and improve usability.
4. Dashboard analytics will provide fast decision support for organizers.
5. Payment and booking screens will minimize steps to reduce checkout drop-off.

6. Functional Requirements

6.1 Core Features (Mandatory)

1. User Authentication using JWT

2. Role-based access for Admin and User
3. Complete CRUD operations for events, venues, categories, tickets, and announcements
4. Search functionality for events, bookings, and users
5. Filtering functionality by category, date, location, ticket price, booking status, and payment status
6. External API integration for payments (Stripe), venue/location assistance (Google Maps), and emails (SendGrid)

6.2 Optional Features

1. Chatbot integration using OpenAI API or Gemini API for FAQs and event recommendations
2. Real-time booking and seat update notifications using Socket.io
3. QR code ticket generation and verification
4. Refund request workflow

7. Frontend Design Details

Component Structure (Brief Hierarchy)

1. App
2. Auth Routes / Protected Routes
3. Shared Layouts (Navbar, Sidebar, Footer)
4. Pages (Home, Events, Details, Checkout, Dashboard, Bookings, Reports)
5. Reusable Components (EventCard, FilterBar, SearchInput, Table, Modal, FormField, StatsCard, PaymentSummary)

State Management Approach

Redux Toolkit will manage authentication state, current user role, event listing filters, booking data, and admin dashboard data. Component-level local state will be used for form handling and lightweight UI behavior.

API Integration Strategy (Frontend side)

Axios will be used to communicate with backend REST APIs. Protected endpoints will use JWT stored securely on the client side according to project preference (HTTP-only cookie recommended, localStorage acceptable for academic prototype). Stripe Checkout session creation will be triggered from the frontend but completed through backend validation.

8. Backend Design Details

API Architecture (RESTful endpoints)

The backend will follow a modular REST architecture with separate route groups for authentication, users, events, venues, categories, bookings, payments, and notifications. Controllers and services will separate business logic from request handling.

Authentication & Authorization Strategy

JWT-based authentication will be implemented for login and protected access. Role-based authorization middleware will restrict admin-only routes such as event creation, revenue reports, and booking management.

Middleware Usage

1. Auth middleware for verifying JWT
2. Role middleware for admin access control
3. Error-handling middleware for centralized API errors
4. Validation middleware for request body checking
5. Multer middleware for image upload if event banners are supported
6. Logging middleware for debugging and monitoring

Error Handling Approach

The application will use standardized JSON error responses, try-catch blocks in async controllers, validation error reporting, and status codes such as 200, 201, 400, 401, 403, 404, and 500. Frontend alerts and toasts will display meaningful user feedback.

9. API Design

Method	Endpoint	Description
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/events	Fetch all events
GET	/api/events/:id	Fetch single event
POST	/api/events	Create event (admin)
PUT	/api/events/:id	Update event (admin)
DELETE	/api/events/:id	Delete event (admin)
GET	/api/categories	Fetch categories
POST	/api/categories	Create category (admin)
GET	/api/venues	Fetch venues
POST	/api/venues	Create venue (admin)
GET	/api/bookings/my	Get logged-in user bookings
POST	/api/bookings	Create provisional booking before payment
POST	/api/payments/create-checkout-session	Create Stripe Checkout session
POST	/api/payments/webhook	Handle Stripe webhook updates
GET	/api/admin/reports	Fetch admin analytics and revenue reports

10. Database Design

10.1 Collections Overview

1. Users

2. Events
3. Categories
4. Venues
5. Bookings
6. Payments
7. Announcements

10.2 Data Dictionary (MANDATORY)

Users Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier
name	String	User full name
email	String	Unique email address
password	String	Encrypted password
role	String	admin or user
phone	String	Contact number
createdAt	Date	Registration timestamp

Events Collection

Field Name	Data Type	Description
_id	ObjectId	Unique event ID
title	String	Event title
description	String	Event details
categoryId	ObjectId	Reference to category
venueId	ObjectId	Reference to venue
date	Date	Event date and time
bannerImage	String	Image URL
ticketPrice	Number	Base ticket price
availableSeats	Number	Remaining seats
status	String	draft / published / cancelled
createdBy	ObjectId	Admin who created event
createdAt	Date	Timestamp

Bookings Collection

Field Name	Data Type	Description
_id	ObjectId	Unique booking ID
userId	ObjectId	Reference to attendee

eventId	ObjectId	Reference to event
ticketQuantity	Number	Number of tickets
totalAmount	Number	Total payable amount
paymentStatus	String	pending / paid / failed / refunded
bookingStatus	String	reserved / confirmed / cancelled / attended
stripeSessionId	String	Linked Stripe checkout session
createdAt	Date	Booking timestamp

11. CRUD Operations Mapping

Operation	Description	API Endpoint
Create	Add new event / venue / category / booking	POST /api/...
Read	View records and lists	GET /api/...
Update	Modify existing event or related record	PUT /api/.../:id
Delete	Remove event, category, or venue	DELETE /api/.../:id

12. Search & Filtering Design

Search Fields: Event title, category name, venue name, attendee name, booking ID, payment status

Filtering Criteria: Date, category, city/location, price range, booking status, event status, payment status

Implementation Approach: Both frontend and backend. Frontend will manage filter UI/state, while backend will process query parameters for efficient MongoDB search and filtering.

13. External API Integration

Primary External Integrations

1. API Name: Stripe Checkout API
2. Purpose: Secure online payment for users buying event tickets
3. Endpoints Used: Backend creates Checkout Session; webhook endpoint updates booking/payment records
4. How Data Will Be Used: Payment amount, session ID, payment status, and booking confirmation
5. API Name: Google Maps Places API
6. Purpose: Venue search, location autocomplete, and better user navigation
7. Endpoints Used: Place autocomplete / place details integration
8. How Data Will Be Used: Venue forms in admin panel and event detail screens in user panel
9. API Name: SendGrid Email API
10. Purpose: Admin-triggered broadcast emails and automatic booking confirmation emails
11. Endpoints Used: Transactional send endpoint via backend service

12. How Data Will Be Used: Confirmation emails, reminders, cancellation alerts, organizer announcements

14. Optional Chatbot Integration

1. Tool/Service Used: OpenAI API or Gemini API
2. Use Case: Event FAQs, ticketing help, event recommendation assistant, and support automation

15. Timeline & Milestones

Week	Task
Week 1	Proposal finalization, requirement analysis, Figma design
Week 2	Backend setup, database schemas, auth APIs, event/category/venue CRUD
Week 3	Frontend development for user and admin panels
Week 4	Integration of frontend + backend + Stripe + Google Maps + SendGrid
Week 5	Testing, bug fixing, polishing, report/final presentation preparation

16. Expected Outcomes

1. Fully functional MERN application for event management and ticket booking
2. Responsive UI with proper UX for both users and admins
3. Secure backend with JWT-based authentication and role authorization
4. Working Stripe payment integration
5. Working venue/location assistance and email notification integrations
6. Complete CRUD + Search + Filtering implementation

17. Contribution Breakdown

Member	Responsibilities
Jalal Haider (i228755)	Frontend lead: React UI, user panel, admin dashboard UI, Redux state, search/filter components, Figma assistance
Umer Abdullah (i221349)	Backend lead: Node/Express APIs, MongoDB schemas, JWT auth, Stripe integration, admin analytics, deployment/testing

18. References

1. Stripe Checkout and Payment APIs - https://docs.stripe.com/
2. Google Maps Places API - https://developers.google.com/maps/
3. SendGrid Email API - https://sendgrid.com/docs/
4. MongoDB Documentation - https://www.mongodb.com/docs/
5. Express.js Documentation - https://expressjs.com/

6. React Documentation - https://react.dev/